



Mini Banco
ESCOM · IPN

PROYECTO FINAL · SISTEMAS DISTRIBUIDOS

Manual técnico

Arquitectura, capa HTTP sobre NIO, replicación líder-réplica, recuperación desde Cloud Storage, generador de carga, rendimiento, despliegue y pruebas del Mini Banco Concurrente Distribuido en la Nube.

EQUIPO 17

Eduardo Alonso Sánchez 7CM3

Vania Ivonne Arévalo del Ángel 7CM4

Miriam Guadalupe Ramírez Sánchez 7CM2

M. en C. Ukranio Coronilla · Junio 2026

Índice

1. Arquitectura general	4
1.1. Vision general: tres nodos con lider fijo	4
1.2. Modelo de datos en memoria	5
1.3. Cableado del servidor en BancoHttp	6
1.3.1. Flujo de alto nivel de una peticion	7
1.4. Estructura de paquetes	7
2. Servidor HTTP sobre NIO	8
2.1. Arquitectura multi-reactor	8
2.2. El bucle del reactor	9
2.3. Camino caliente e <i>inline</i> contra worker	11
2.4. Capa HTTP/1.1: parser, modelo y ruteo	11
3. API y autenticacion	13
3.1. Endpoints REST	13
3.2. Formatos JSON de cuentas y transferencias	14
3.3. Autenticacion: JWT y bcrypt	15
3.4. Validacion del Bearer y cache de tokens	16
4. Concurrencia y transferencias	17
4.1. Dinero en centavos: exactitud y conservacion	17
4.2. Estructura concurrente del repositorio	18
4.3. transferir(): atomicidad sin interbloqueo	18
4.4. Errores de negocio y mapeo a codigos REST	20
5. Replicacion y tolerancia a fallos	21
5.1. Log por numero de secuencia	21
5.2. El lider: catch-up y streaming TCP	22
5.3. La replica: aplicar y reconectar	23
5.4. Punto de control local y reconciliacion por RESET	24
5.5. Rol por entorno y resincronizacion ante fallos	25
6. Persistencia y recuperacion en Cloud Storage	26
6.1. Escritura asincrona que no bloquea la transferencia	26
6.2. El subidor real: solo JDK contra la API JSON de GCS	27
6.3. Siembra del contador y recuperacion en frio	28
6.4. Exposicion de la metrica en el dashboard	30
7. Generador de carga y rendimiento	30
7.1. Modelo de carga	30
7.2. Motor asincrono con ventana en vuelo	31
7.3. Parametro clientes (K) y verificacion del invariante	31
7.4. Resultados	32
8. Dashboard de observabilidad	33
8.1. Pagina servida en GET /	33

8.2. Estado de un nodo: GET /stats	33
8.3. Vista agregada del cluster: GET /panel	34
8.4. Refresco reactivo en el navegador	35
9. Despliegue en Google Cloud Platform	35
9.1. Topologia y firewall	36
9.2. Despliegue sin SSH via Cloud Storage y systemd	36
9.3. Configuración por variables de entorno	37
9.4. Operación del punto de control en las replicas	39
10. Pruebas y calidad	40
10.1. El harness: TestRunner y MiniTest	40
10.2. Como se ejecutan	41
10.3. Que cubre cada suite	41

1 Arquitectura general

El Mini Banco Concurrente Distribuido es un sistema de tres nodos que mantiene en memoria un conjunto de cuentas y atiende cuatro operaciones bancarias por HTTP. El estado no vive en ningún gestor de base de datos: las cuentas se cargan desde un CSV al arrancar y residen en estructuras concurrentes del proceso Java. Esta sección describe la topología de los nodos, el modelo de datos, el cableado del servidor en `BancoHttp.crear()` y la organización de los paquetes.

1.1 Vision general: tres nodos con líder fijo

El despliegue usa tres procesos idénticos: `nodo-1` como líder fijo y `nodo-2` y `nodo-3` como réplicas. Todas las lecturas y escrituras de la lógica bancaria se dirigen al líder; las réplicas siguen al líder y aplican sus transacciones en vivo, de modo que conservan una copia consistente del estado sin atender la operación bancaria directamente.

El rol no se compila ni se configura en código: lo decide la variable de entorno `BANCO_LIDER_HOST`. Si no está definida, el nodo es LÍDER; si está definida, el nodo es REPLICA y apunta a ese host. El criterio vive en `server/MainServer.java`:

Listing 1: Elección de rol por variable de entorno (MainServer.java)

```
/** LIDER si no hay BANCO_LIDER_HOST; en otro caso REPLICA. */
private static boolean esLider() {
    String liderHost = System.getenv("BANCO_LIDER_HOST");
    return liderHost == null || liderHost.isBlank();
}
```

El mismo método gobierna el arranque de la replicación: el líder levanta un `ServidorReplicacion` en un puerto TCP, mientras que la réplica crea un `ClienteReplicacion` que se conecta al líder para recibir y aplicar las transacciones.

Listing 2: Arranque de replicación según el rol (MainServer.java)

```
if (esLider()) {
    new ServidorReplicacion(CuentaRepository.get(), puertoRepl).iniciar();
    System.out.println("Rol: LIDER - replicacion TCP en puerto " + puertoRepl);
} else {
    String liderHost = System.getenv("BANCO_LIDER_HOST");
    new ClienteReplicacion(CuentaRepository.get(), liderHost, puertoRepl).iniciar();
    System.out.println("Rol: REPLICA - siguiendo a " + liderHost + ":" + puertoRepl);
}
```

NOTA

Solo el líder activa el log durable en Cloud Storage (variable `BANCO_GCS_BUCKET`), y lo hace ANTES de servir tráfico para poder recuperar el estado reaplicando el log sobre las cuentas del CSV en caso de que todos los nodos hubieran caído.

1.2 Modelo de datos en memoria

El dataset son aproximadamente 820,000 cuentas que se cargan desde `alumnos.csv` al iniciar el nodo. La carga la hace `data/AlumnosLoader.java`, que lee el archivo línea por línea: el `id` de cada cuenta es el número de línea en orden de lectura empezando en 1. El CSV no tiene cabecera, por lo que la primera fila ya es `id=1`.

Una decisión central del modelo es guardar el dinero SIEMPRE en centavos como `long`, nunca como `double`, para que el invariante de suma constante de saldos se mantenga exacto. El loader convierte el saldo decimal del CSV a centavos enteros:

Listing 3: Carga de cuentas con `id` base 1 y saldo en centavos (`AlumnosLoader.java`)

```
id++; // base 1, en orden de lectura
long centavos = new BigDecimal(c[3].trim())
    .movePointRight(2)
    .setScale(0, RoundingMode.HALF_UP)
    .longValueExact();
repo.put(new Cuenta(id, c[0].trim(), c[1].trim(), c[2].trim(), centavos));
```

La entidad es `model/Cuenta.java`. Expone `id`, los datos del propietario y el saldo en centavos, y trae un `ReentrantLock` por cuenta que las transferencias toman en orden de `id` para evitar interbloqueos. El saldo se ofrece como decimal exacto solo al construir el JSON de salida, via `saldoDecimal()`, que divide los centavos entre 100 con `BigDecimal`.

Listing 4: Saldo en centavos y conversión a decimal exacto (`Cuenta.java`)

```
private volatile long saldoCentavos;

// Lock por cuenta para transferencias concurrentes (se toma en orden de id).
public final ReentrantLock lock = new ReentrantLock();

/** Saldo como decimal exacto (centavos / 100) para el JSON de salida. */
public BigDecimal saldoDecimal() {
    return BigDecimal.valueOf(saldoCentavos, 2);
}
```

Cada transferencia confirmada se representa como `model/Transaccion.java`, un record inmutable con un número de secuencia que impone un orden total sobre las operaciones; ese `seq` es la base de la replicación y del log durable.

Listing 5: Transacción con orden total (`Transaccion.java`)

```
/** Una transferencia registrada con su número de secuencia (orden total). */
public record Transaccion(long seq, int origenId, int destinoId, long montoCentavos) {
}
```

El almacén es `data/CuentaRepository.java`, un singleton sin SGBD que guarda las cuentas en un `ConcurrentHashMap` y lleva la secuencia con un `AtomicLong`. Su método `transferir()` es atómico:

valida monto y cuentas, bloquea las dos cuentas en orden de `id`, mueve el saldo, asigna el `seq` y notifica al observador del log durable.

Listing 6: Bloqueo ordenado y conmutacion atomica del saldo (CuentaRepository.java)

```
// Bloqueo en orden de id (menor primero) para no provocar deadlock.
Cuenta primero = origenId < destinoId ? origen : destino;
Cuenta segundo = origenId < destinoId ? destino : origen;
primero.lock.lock();
segundo.lock.lock();
try {
    if (origen.getSaldoCentavos() < montoCentavos) {
        return Resultado.de(Estado.SALDO_INSUFICIENTE);
    }
    origen.setSaldoCentavos(origen.getSaldoCentavos() - montoCentavos);
    destino.setSaldoCentavos(destino.getSaldoCentavos() + montoCentavos);
    long seq = secuencia.incrementAndGet();
    // ...
}
```

IMPORTANTE

No hay base de datos. Todo el estado es volátil: si el proceso del líder termina, la única copia durable es el log en Cloud Storage. Las cuentas creadas en caliente vía `/api/register` que no estén en el CSV no se recuperan tras un reinicio en frío.

1.3 Cableado del servidor en BancoHttp

El servidor HTTP de cada nodo se arma en `server/BancoHttp.java`, mediante `crear()`, compartido por `MainServer` y por las pruebas. Ahí se registra cada ruta con su *handler* y se indica en qué reactor se ejecuta: las cuatro operaciones del banco se sirven `INLINE` en el hilo de I/O salvo `/api/login` y `/api/register` (`bcrypt`) y `/panel` (consulta a los peers), que van al pool `WORKER` por ser bloqueantes.

Listing 7: Registro de rutas, handlers y reactores (BancoHttp.java)

```
Ruteador r = new Ruteador()
    .registrar("/api/register", auth, Ruteador.WORKER)
    .registrar("/api/login", auth, Ruteador.WORKER)
    .registrar("/api/accounts", new AccountHandler(), Ruteador.INLINE)
    .registrar("/api/transactions/transfer", new TransferHandler(), Ruteador.INLINE)
    .registrar("/stats", new StatsHandler(), Ruteador.INLINE)
    .registrar("/panel", new PanelHandler(idLocal, peers), Ruteador.WORKER)
    .registrar("/", new PaginaHandler(), Ruteador.INLINE);
return new ServidorNio(puerto, r, reactores(), hilosWorker());
```

El número de reactores (hilos de I/O) y de hilos worker se calcula a partir de los núcleos disponibles y se puede ajustar con `BANCO_REACTORES` y `BANCO_WORKERS`; en una máquina de 4 vCPU se usan 3 reactores por defecto.

1.3.1 Flujo de alto nivel de una petición

Una petición al líder recorre las siguientes etapas:

- `MainServer` carga el CSV con `AlumnosLoader.cargar()` y crea el servidor con `BancoHttp.crear()`, que devuelve un `ServidorNio`.
- El `ServidorNio` acepta la conexión en un reactor; el `Ruteador` elige el handler por el prefijo de la ruta y lo ejecuta `INLINE` o en el pool `WORKER` según lo registrado.
- El handler (p. ej. `TransferHandler`) parsea el cuerpo con `json/Json.java`, opera sobre `CuentaRepository.get()` y, en el caso de transferir, dispara `transferir()`, que asigna el `seq` y notifica al observador del log durable.
- La respuesta vuelve serializada como JSON con `Json.toJson()`.

El parseo y la serialización JSON se centralizan en `json/Json.java`, que envuelve a `gson`:

Listing 8: Fachada JSON centralizada (`Json.java`)

```
public static JsonObject parse(String body) {  
    return JsonParser.parseString(body).getAsJsonObject();  
}  
  
public static String toJson(Object o) {  
    return GSON.toJson(o);  
}
```

1.4 Estructura de paquetes

El código vive bajo `com.escom.banco` con esta separación de responsabilidades:

Paquete	Responsabilidad
model	Entidades de dominio: Cuenta y Transaccion.
data	Almacen en memoria y carga: CuentaRepository, AlumnosLoader, RegistroTransacciones.
server	Nodo HTTP y handlers de las rutas (MainServer, BancoHttp, los *Handler).
server.http	Capa HTTP propia: Ruteador, ParserHttp, Solicitud, Respuesta.
server.nio	Servidor no bloqueante: ServidorNio, Reactor, Conexion.
replicacion	Lider/replica por TCP: ServidorReplicacion, ClienteReplicacion, CodecTransaccion.
almacen	Log durable en Cloud Storage: AlmacenGcs, GcsSubidor.
auth	Identidad y JWT: AuthService, JWTUtil, PasswordUtil.
json	Fachada de serializacion sobre gson: Json.
carga	Generador de carga para pruebas: GeneradorCarga.

2 Servidor HTTP sobre NIO

El banco no usa `com.sun.net.httpserver`: implementa su propio servidor HTTP/1.1 sobre `java.nio`, con cero dependencias externas. El nucleo es un modelo *multi-reactor*: varios hilos, cada uno con un `Selector` y un `ServerSocketChannel` propios, multiplexan miles de conexiones sin un hilo por socket. El camino caliente (consultas y operaciones del banco) se atiende y se escribe sin salir del hilo del reactor; solo las rutas que bloquean (bcrypt en login/registro, llamadas a peers en el panel) se descargan a un pool worker. La capa HTTP la forman un parser sin estado y un puñado de objetos inmutables.

2.1 Arquitectura multi-reactor

La clase `server/nio/ServidorNio.java` orquesta el arranque. Su constructor recibe el puerto, el `Ruteador`, el numero de reactores y el tamaño del pool worker; ambos numeros son configurables y se fuerzan a un minimo de uno. El metodo `iniciar()` es idempotente y `synchronized`: crea el pool de hilos daemon, instancia los reactores y decide como repartir las conexiones segun haya o no soporte de `SO_REUSEPORT`.

Hay tres ramas de *bind*. Con un solo reactor, un unico `ServerSocketChannel` se registra en ese reactor. Con varios reactores y `SO_REUSEPORT` disponible, cada reactor abre su propio socket en el mismo puerto y es el kernel quien reparte las conexiones entrantes. Si no hay `SO_REUSEPORT`, un unico acceptor acepta y reparte los canales round-robin entre todos los reactores, incluido el propio acceptor (`reactores[0].conAcceptor(s, reactores)` pasa el arreglo completo como destinos del reparto).

Listing 9: Tres modos de reparto en `ServidorNio.iniciar()`


```

boolean reuseport = numReactores > 1 && soportaReuseport();
if (numReactores == 1) {
    ServerSocketChannel s = abrir(puerto, false);
    reactores[0].conAcceptor(s, null);
} else if (reuseport) {
    for (int i = 0; i < numReactores; i++) {
        ServerSocketChannel si = abrir(puerto, true);
        reactores[i].conAcceptor(si, null);
    }
} else {
    ServerSocketChannel s = abrir(puerto, false);
    reactores[0].conAcceptor(s, reactores); // acceptor unico round-robin
}

```

El metodo `abrir()` configura el socket en modo no bloqueante, activa `SO_REUSEADDR`, agrega `SO_REUSEPORT` cuando corresponde y hace `bind` con un `backlog` de 1024. La detección de `SO_REUSEPORT` es defensiva: `soportaReuseport()` consulta `supportedOptions()` de un canal de prueba, porque esa opcion no existe en todos los sistemas operativos.

Listing 10: Apertura no bloqueante del ServerSocketChannel

```

ServerSocketChannel s = ServerSocketChannel.open();
s.configureBlocking(false);
s.setOption(StandardSocketOptions.SO_REUSEADDR, true);
if (reuseport) s.setOption(StandardSocketOptions.SO_REUSEPORT, true);
s.bind(new InetSocketAddress(puerto), BACKLOG);

```

NOTA

Cada reactor corre en un hilo daemon nombrado `banco-reactor-i`; el pool worker usa hilos `banco-worker-n`. Tanto `detener()` como el rollback `limpiarParcial()` cierran selectores, sockets y el pool de forma ordenada.

2.2 El bucle del reactor

La clase `server/nio/Reactor.java` es el corazon del servidor: un hilo, un selector y un bucle estilo *epoll*. En cada vuelta primero registra los canales nuevos que pudo haberle pasado el acceptor (cola nuevos), luego drena la cola de escrituras pendientes encoladas por los workers (pendientesEscritura) y solo entonces hace `select()` para atender los eventos `OP_ACCEPT`, `OP_READ` y `OP_WRITE`.

Listing 11: Bucle principal del Reactor

```

while (activo) {
    SocketChannel ch;
    while ((ch = nuevos.poll()) != null) registrarCanal(ch);
    SelectionKey wk;
    while ((wk = pendientesEscritura.poll()) != null) {

```

```

        if (wk.isValid()) flush(wk);
    }
    selector.select();
    Iterator<SelectionKey> it = selector.selectedKeys().iterator();
    while (it.hasNext()) {
        SelectionKey key = it.next();
        it.remove();
        if (key.isAcceptable()) { if (activo) aceptar(); }
        else if (key.isReadable()) leer(key);
        else if (key.isWritable()) flush(key);
    }
}

```

IMPORTANTE

La INVARIANTE central del diseño: **solo el hilo del reactor toca el `SocketChannel` y `key.interestOps()`**. Los workers nunca escriben al canal; unicamente encolan bytes en la `Conexion` y la `key` en pendientesEscritura, y despiertan al selector. Romper esta invariante introduciría carreras imposibles de depurar sobre el estado del socket.

El estado por conexión vive en `server/nio/Conexion.java`, que se adjunta como *attachment* de la `SelectionKey`. Contiene un buffer de lectura acumulativo (`buf`, `len`) que crece por duplicación hasta un tope, y una cola de salida `salida` de `ByteBuffer`. La cola es `ConcurrentLinkedQueue` precisamente porque un worker puede depositar la respuesta desde otro hilo, pero la escritura al canal sigue siendo exclusiva del reactor.

Al aceptar, `registrarCanal()` pone el canal en no bloqueante, activa `TCP_NODELAY` y lo registra para `OP_READ` con una `Conexion` nueva. La lectura (`leer()`) asegura espacio, lee del canal al buffer, despacha con `procesar()` y vacía con `flush()`. El `flush()` aplica *back-pressure*: si un `ByteBuffer` no se escribió completo, registra interés en escritura y espera; cuando termina de vaciar vuelve a `OP_READ` y, si quedan bytes en el buffer, reanuda el parseo (clave para keep-alive y pipelining).

Listing 12: flush con back-pressure y reanudación del parseo

```

while ((bb = cx.salida.peek()) != null) {
    ch.write(bb);
    if (bb.hasRemaining()) {
        key.interestOps(SelectionKey.OP_READ | SelectionKey.OP_WRITE);
        return;
    }
    cx.salida.poll();
}
if (cx.cerrarTrasFlush) { cerrar(key); return; }
key.interestOps(SelectionKey.OP_READ);
if (cx.len > 0 && !cx.esperandoWorker) {
    procesar(key, cx);
    if (!cx.salida.isEmpty()) continue;
}

```

2.3 Camino caliente e inline contra worker

El metodo `procesar()` parsea y despacha todas las peticiones completas que haya en el buffer. Resuelve la ruta con el `Ruteador` y bifurca segun como este marcada. Una ruta `INLINE` (camino caliente: consultar cuentas, transferir, estadisticas, pagina) se ejecuta y se serializa en el mismo hilo del reactor, y el bucle sigue drenando posibles peticiones pipelined. Una ruta `WORKER` marca `esperandoWorker` y hace `return`: deja de parsear hasta que llegue la respuesta, porque estas peticiones no se pipelinean.

Listing 13: Bifurcacion inline vs worker en `procesar()`

```
if (ruta.worker) {
    cx.esperandoWorker = true;
    despacharWorker(key, cx, ruta.manejador, sol, cerrar);
    return; // no parseamos mas hasta responder
}
Respuesta resp = ejecutar(ruta.manejador, sol);
boolean cerrarFinal = cerrar || resp.cerrar;
cx.encolar(Conexion.serialize(resp, cerrarFinal));
if (cerrarFinal) { cx.cerrarTrasFlush = true; return; }
```

El retorno de un worker usa **dos** estructuras concurrentes, no una. El worker ejecuta el handler, serializa la respuesta y la encola en `Conexion.salida`; ademas marca la key en la cola `pendientesEscritura` del reactor y dispara `selector.wakeup()` mediante `marcarPendiente()`. Asi el reactor, al volver del `select()`, encuentra la key pendiente y hace el `flush()`; nunca es el worker quien escribe al socket.

Listing 14: `despacharWorker`: el worker solo encola y despierta

```
pool.execute(() -> {
    Respuesta resp = ejecutar(m, sol);
    boolean cerrarFinal = cerrar || resp.cerrar;
    cx.encolar(Conexion.serialize(resp, cerrarFinal));
    if (cerrarFinal) cx.cerrarTrasFlush = true;
    cx.esperandoWorker = false;
    marcarPendiente(key); // encola la key + selector.wakeup()
});
```

CONSEJO

La distincion inline contra worker la decide el `Ruteador` al registrar, no el handler. Mantener los handlers triviales en el reactor evita el coste de un *handoff* de hilo por cada peticion; reservar el pool para lo que de verdad bloquea (bcrypt, E/S de red) protege a los reactores de quedarse atascados.

2.4 Capa HTTP/1.1: parser, modelo y ruteo

El parser `server/http/ParserHttp.java` es **sin estado**: cada intento re-escanea el rango `[0, len)` del buffer y devuelve un `Resultado` con estado `LISTA` (con la `solicitud` y cuantos bytes consumir),

INCOMPLETA (faltan datos) o ERROR. Busca el fin de cabeceras (CRLF o LFLF), interpreta la *request-line*, normaliza las cabeceras a minúsculas (gana la primera en duplicados) y respeta el Content-Length. Aplica límites de tamaño: si las cabeceras superan MAX_HEADERS responde 431, y un Content-Length mayor que MAX_CUERPO es 400.

Listing 15: Decision keep-alive del parser (Content-Length y Connection)

```
boolean expectContinue = "100-continue".equalsIgnoreCase(headers.get("expect"));
int disponibles = len - bodyStart;
if (disponibles < cl) return Resultado.incompleta(expectContinue);
// ...
String conn = headers.get("connection");
boolean pideCerrar = (conn != null && conn.toLowerCase().contains("close"))
    || (http10 && (conn == null || !conn.toLowerCase().contains("keep-alive")));
```

El parser también sostiene keep-alive y pipelining: indica con bytesConsumidos cuanto compactar para que el resto del buffer sea el inicio de la siguiente petición, y emite 100 Continue cuando una petición incompleta trae Expect: 100-continue. Antes de rutear, normalizarPath() quita esquema, autoridad, query y fragment del request-target, dejando solo el path limpio.

El modelo de datos es inmutable. server/http/Solicitud.java expone metodo(), path(), un header() insensible a mayúsculas y el cuerpo en bytes o texto. server/http/Respuesta.java ofrece fabricas json(), html(), sinCuerpo() y error(); calcula el Content-Length sobre los bytes UTF-8 del cuerpo, nunca sobre String.length(), porque un Content-Length incorrecto desincroniza el keep-alive. El server/http/Manejador.java es una interfaz funcional pura Solicitud a Respuesta: no conoce el SocketChannel ni los interestOps.

Listing 16: Manejador: handler como función pura

```
@FunctionalInterface
public interface Manejador {
    Respuesta manejar(Solicitud s);
}
```

Finalmente, server/http/Ruteador.java replica el *longest-prefix* de HttpServer.createContext: registrar() asocia un prefijo de path a un Manejador y a la marca INLINE o WORKER, y al registrar mantiene la tabla ordenada por prefijo más largo primero. Así resolver() solo barre las entradas en ese orden y devuelve el **primer** prefijo que sea prefijo del path, que por construcción es el más largo; ese corte temprano abarata el camino caliente. La validación del verbo HTTP (responder 405) queda en cada handler; aquí solo se rutea por path.

Listing 17: Orden en registrar() y primer match en resolver()

```
public Ruteador registrar(String prefijo, Manejador m, boolean worker) {
    entradas.add(new Entrada(prefijo, new Ruta(m, worker)));
    // Prefijo mas largo primero (una vez, en el arranque).
    entradas.sort((a, b) -> b.prefijo.length() - a.prefijo.length());
    return this;
}
```

```
public Ruta resolver(String path) {
    for (Entrada e : entradas) {
        if (path.startsWith(e.prefijo)) return e.ruta; // ya ordenadas desc
    }
    return null;
}
```

3 API y autenticación

El nodo expone una API REST sobre HTTP con exactamente cuatro endpoints funcionales, mas un `/stats` de monitoreo. Las rutas se declaran en `server/BancoHttp.java` y cada una resuelve con un manejador propio. El registro y el inicio de sesión crean credenciales y emiten un JWT; las operaciones sobre cuentas y transferencias exigen ese token en cada petición.

3.1 Endpoints REST

Los cuatro endpoints del proyecto son: `POST /api/register` para crear una credencial, `POST /api/login` que devuelve un JWT, `GET /api/accounts/{id}` para consultar una cuenta y `POST /api/transactions/transfer` para mover saldo. Los dos últimos exigen el header `Authorization: Bearer <token>`.

Ruta	Metodo	JWT	Manejador
<code>/api/register</code>	POST	No	AuthHandler
<code>/api/login</code>	POST	No	AuthHandler
<code>/api/accounts/{id}</code>	GET	Si	AccountHandler
<code>/api/transactions/transfer</code>	POST	Si	TransferHandler

El registro (`server/AuthHandler.java`) recibe `username` y `password`, los valida y delega en `AuthService`. Devuelve 201 si crea la credencial o 409 si el usuario ya existe.

Listing 18: Registro de credenciales en AuthHandler

```
private Respuesta registrar(Solicitud s) {
    JsonObject in = parse(s);
    if (in == null) return Respuesta.error(400, "JSON inválido");
    if (!in.has("username") || !in.has("password")) {
        return Respuesta.error(400, "Faltan username/password");
    }
    boolean creado = auth.register(in.get("username").getAsString(),
                                   in.get("password").getAsString());
    if (creado) return Respuesta.json(201, "{\"status\":\"registrado\"}");
    return Respuesta.error(409, "Usuario ya existe");
}
```

El login valida las mismas dos llaves y, si las credenciales son correctas, devuelve el token; si no, responde 401.

Listing 19: Respuesta de POST /api/login

```
{ "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.<payload>.<firma>" }
```

3.2 Formatos JSON de cuentas y transferencias

server/AccountHandler.java extrae el id del ultimo segmento de la ruta (/api/accounts/{id}) sin `split('/')` (con `Long.parseLong` sobre indices, para no alocar un `String[]` por peticion), consulta el `CuentaRepository` y arma el JSON a mano (sin gson) por ser la ruta caliente. La respuesta lleva id, propietario y balance. Un id no numerico responde 400, pero un id numerico fuera de `[1, Integer.MAX_VALUE]` es un recurso inexistente: responde 404.

Listing 20: Construcción de la respuesta de cuenta

```
long idLargo;
try { idLargo = Long.parseLong(path, ini, fin, 10); }
catch (NumberFormatException e) { return Respuesta.error(400, "Id invalido"); }
if (idLargo < 1 || idLargo > Integer.MAX_VALUE) return Respuesta.error(404, "Cuenta no
    ↪ encontrada");

Cuenta c = repo.get((int) idLargo);
if (c == null) return Respuesta.error(404, "Cuenta no encontrada");

// JSON en un solo StringBuilder: propietario precomputado y saldo desde
// long (sin BigDecimal), para minimizar allocations por GET.
StringBuilder sb = new StringBuilder(96);
sb.append("{ \"id\": ").append(c.id)
    .append(", \"propietario\": \"").append(c.propietario)
    .append("\", \"balance\": ");
c.appendSaldo(sb);
sb.append('}');
return Respuesta.json(200, sb.toString());
```

Listing 21: Respuesta de GET /api/accounts/125

```
{ "id": 125, "propietario": "...", "balance": 15750.25 }
```

La transferencia (server/TransferHandler.java) espera en el cuerpo `sourceAccountId` y `targetAccountId` como cadenas y `amount` como decimal. El monto se convierte a centavos con `BigDecimal` para evitar errores de coma flotante antes de invocar `CuentaRepository.transferir()`.

Listing 22: Parseo de ids y monto en TransferHandler

```
origen = Integer.parseInt(in.get("sourceAccountId").getAsString().trim());
destino = Integer.parseInt(in.get("targetAccountId").getAsString().trim());
```

```
centavos = new BigDecimal(in.get("amount").getAsString().trim())
    .movePointRight(2)
    .setScale(0, RoundingMode.HALF_UP)
    .longValueExact();
```

Listing 23: Cuerpo de POST /api/transactions/transfer

```
{ "sourceAccountId": "123", "targetAccountId": "456", "amount": 200.00 }
```

El resultado del repositorio se traduce a códigos REST: 200 con `seq` si la operación es correcta, 404 si una cuenta no existe y 400 para misma cuenta, monto inválido o saldo insuficiente.

Código	Significado en la API
200	Operación correcta (consulta o transferencia).
201	Credencial registrada.
400	JSON o datos inválidos, faltan campos.
401	Sin JWT válido o credenciales incorrectas.
404	Cuenta no encontrada.
409	Usuario ya existe.

NOTA

`server/StatsHandler.java` sirve `GET /stats` sin JWT: es solo monitoreo de un nodo y entrega el saldo total en centavos para verificar el invariante de conservación en una sola petición.

3.3 Autenticación: JWT y bcrypt

Las credenciales viven aparte de las cuentas: `auth/User.java` guarda `username` y el hash de la contraseña, y `auth/UserRepository.java` las almacena en un `ConcurrentHashMap` usando `saveIfAbsent()` para evitar duplicados. `auth/AuthService.java` coordina ambos pasos: hacea al registrar y verifica al iniciar sesión antes de emitir el token.

Listing 24: Flujo de login en AuthService

```
public String login(String username, String password) {
    User u = repo.findByUsername(username);
    if (u == null) return null;
    if (!PasswordUtil.verify(password, u.getPasswordHash())) return null;
    return JWTUtil.generateToken(username);
}
```

El hashing usa bcrypt mediante la librería jbcrypt en `auth/PasswordUtil.java` (`BCrypt.hashpw` con `BCrypt.gensalt` al guardar y `BCrypt.checkpw` al verificar). Es un algoritmo deliberadamente lento

(50-100 ms), por eso `register` y `login` corren en el pool `WORKER` y no en el reactor.

El token se emite y valida con HMAC256 (librería `com.auth0 java-jwt`) en `auth/JWTUtil.java`. El secreto y el issuer son fijos para que un token emitido por el líder valide en las réplicas; el secreto se toma de `BANCO_JWT_SECRET`.

Listing 25: Emisión y verificación del JWT

```
public static String generateToken(String username) {
    return JWT.create()
        .withIssuer(ISSUER)
        .withSubject(username)
        .sign(ALGO);
}

public static DecodedJWT verify(String token) throws JWTVerificationException {
    return JWT.require(ALGO).withIssuer(ISSUER).build().verify(token);
}
```

IMPORTANTE

`BANCO_JWT_SECRET` debe ser idéntico en los tres nodos. Si difiere, un token válido en el líder será rechazado con 401 en las réplicas.

3.4 Validación del Bearer y cache de tokens

Los manejadores protegidos llaman a `HttpUtil.usuarioAutenticado()` (`server/HttpUtil.java`) al inicio: si devuelve `null` responden 401. El método lee el header con la clave en minúscula (`authorization`, ya normalizada por el parser, para no realocar), localiza el primer espacio y compara el scheme con `bearer` de forma case-insensitive (RFC 7235), recorta el token con `trim` y verifica la firma con `JWTUtil`.

Como la validación HMAC se repite en cada petición con el mismo token, los tokens ya verificados se guardan en un `ConcurrentHashMap` (token a subject) con tope de `TOPE_CACHE` entradas, evitando recalcular la firma en la ruta caliente.

Listing 26: Validación del Bearer con cache en `HttpUtil`

```
public static String usuarioAutenticado(Solicitud s) {
    String auth = s.header("authorization");
    if (auth == null) return null;
    int sp = auth.indexOf(' ');
    if (sp < 0) return null;
    if (!auth.substring(0, sp).equalsIgnoreCase("bearer")) return null;
    String token = auth.substring(sp + 1).trim();
    String cacheado = TOKENS_VALIDOS.get(token);
    if (cacheado != null) return cacheado;
    try {
        String sub = JWTUtil.verify(token).getSubject();
        if (TOKENS_VALIDOS.size() < TOPE_CACHE) TOKENS_VALIDOS.put(token, sub);
        return sub;
    } catch (Exception e) {
        return null;
    }
}
```



```

    } catch (Exception e) {
        return null;
    }
}

```

Finalmente, `server/BancoHttp.java` mapea cada ruta a su manejador y decide donde se ejecuta. `/api/accounts` y `/api/transactions/transfer` se sirven `INLINE` en el reactor (son cortas y de alta frecuencia), mientras que `/api/register` y `/api/login` van al `poolWORKER` por el costo de `bcrypt`.

Listing 27: Mapeo de rutas en `BancoHttp`

```

Ruteador r = new Ruteador()
    .registrar("/api/register", auth, Ruteador.WORKER)
    .registrar("/api/login", auth, Ruteador.WORKER)
    .registrar("/api/accounts", new AccountHandler(), Ruteador.INLINE)
    .registrar("/api/transactions/transfer", new TransferHandler(), Ruteador.INLINE)
    .registrar("/stats", new StatsHandler(), Ruteador.INLINE)
// ...

```

4 Concurrencia y transferencias

El núcleo transaccional del mini banco vive en memoria, sin SGBD, como exige el enunciado. La integridad del dinero descansa en dos decisiones de diseño: el saldo se representa siempre en centavos como `long` (nunca `double`) y cada transferencia toma los candados de las dos cuentas en un orden total para evitar interbloqueos. Los componentes implicados son `model/Cuenta.java` (el dato y su candado) y `data/CuentaRepository.java` (la lógica concurrente y el invariante).

4.1 Dinero en centavos: exactitud y conservación

La clase `Cuenta` guarda el saldo en un `long` de centavos. El uso de aritmética entera elimina los errores de redondeo del punto flotante, de modo que sumas y restas son exactas y el invariante de conservación del total se mantiene al centavo. La salida JSON formatea el saldo directo desde el `long` con `appendSaldo(StringBuilder)` (sin crear un `BigDecimal` por GET); `saldoDecimal()` se conserva como conversión exacta de respaldo, escalando el entero por 10^{-2} .

Listing 28: Saldo entero y conversión exacta en `Cuenta.java`

```

private volatile long saldoCentavos;

public long getSaldoCentavos() { return saldoCentavos; }
public void setSaldoCentavos(long c) { this.saldoCentavos = c; }

/** Saldo como decimal exacto (centavos / 100) para el JSON de salida. */
public BigDecimal saldoDecimal() {
    return BigDecimal.valueOf(saldoCentavos, 2);
}

```

Cada `Cuenta` expone además un `ReentrantLock` propio, que es la unidad de exclusión mutua usada por las transferencias. Tener un candado por cuenta (en vez de uno global) permite que transferencias sobre cuentas disjuntas avancen en paralelo.

Listing 29: Candado por cuenta en `Cuenta.java`

```
// Lock por cuenta para transferencias concurrentes (se toma en orden de id).  
public final ReentrantLock lock = new ReentrantLock();
```

4.2 Estructura concurrente del repositorio

`CuentaRepository` es un singleton (`CuentaRepository.get()`) que almacena las cuentas en un `ConcurrentHashMap` indexado por id. El `AtomicLong` `secuencia` es el número de secuencia monótono que ordena las transacciones (base de la replicación y del log en GCS). El conteo de commits `totalTransferencias` es un `LongAdder`: en el hot path hace `increment()` sin CAS global (`sum()` solo lo lee `/stats`), quitando uno de los dos puntos de serialización global por transferencia. El campo `ultimaTxId` es `volatile` para publicar el último seq sin candado adicional.

Listing 30: Estado concurrente de `CuentaRepository.java`

```
private final ConcurrentHashMap<Integer, Cuenta> cuentas = new ConcurrentHashMap<>();  
  
private final AtomicLong secuencia = new AtomicLong(0);  
private final LongAdder totalTransferencias = new LongAdder();  
private volatile long ultimaTxId = 0;  
private final RegistroTransacciones registro = new RegistroTransacciones();  
  
// Serializa secuencia+registro.agregar+observador: lock mas interno (sin deadlock).  
private final ReentrantLock seqLock = new ReentrantLock();
```

El invariante de correctitud es que la suma de todos los saldos permanece constante ante cualquier secuencia de transferencias. El método `saldoTotalCentavos()` lo materializa recorriendo las cuentas y sumando sus centavos; se usa en pruebas para verificar la conservación del dinero.

Listing 31: Verificación del invariante en `CuentaRepository.java`

```
/** Suma de todos los saldos en centavos (para verificar el invariante). */  
public long saldoTotalCentavos() {  
    long total = 0;  
    for (Cuenta c : cuentas.values()) total += c.getSaldoCentavos();  
    return total;  
}
```

4.3 `transferir()`: atomicidad sin interbloqueo

`CuentaRepository.transferir()` es la operación central. Primero rechaza los casos triviales (misma cuenta, monto no positivo) y la ausencia de cualquiera de las cuentas. Acto seguido aplica

la clave anti-deadlock: ordena los dos candados por id (el menor primero) de manera que dos hilos que toquen el mismo par de cuentas siempre los adquieran en el mismo orden, eliminando la espera circular. Bajo ambos candados valida el saldo y mueve el monto; luego, dentro de un tercer candado `seqLock` (el mas interno), asigna el `seq`, registra la `Transaccion` y avisa al observador en un mismo tramo serializado, de modo que el orden de insercion en el registro coincida con el orden de `seq` (evita perder transacciones en la replicacion en vivo). El bloque `finally` garantiza la liberacion en orden inverso.

Listing 32: Bloqueo ordenado y commit en `CuentaRepository.java`

```
// Bloqueo en orden de id (menor primero) para no provocar deadlock.
Cuenta primero = origenId < destinoId ? origen : destino;
Cuenta segundo = origenId < destinoId ? destino : origen;
primero.lock.lock();
segundo.lock.lock();
try {
    if (origen.getSaldoCentavos() < montoCentavos) {
        return Resultado.de(Estado.SALDO_INSUFICIENTE);
    }
    origen.setSaldoCentavos(origen.getSaldoCentavos() - montoCentavos);
    destino.setSaldoCentavos(destino.getSaldoCentavos() + montoCentavos);
    // Tramo serializado: orden de insercion en registro = orden de seq.
    seqLock.lock();
    try {
        long seq = secuencia.incrementAndGet();
        totalTransferencias.increment();
        ultimaTxId = seq;
        Transaccion tx = new Transaccion(seq, origenId, destinoId, montoCentavos);
        registro.agregar(tx);
        observador.accept(tx);
        return Resultado.ok(seq);
    } finally {
        seqLock.unlock();
    }
} finally {
    segundo.lock.unlock();
    primero.lock.unlock();
}
```

NOTA

Como cada cuenta cede el mismo monto que recibe la otra, el delta neto sobre el total es cero: por eso `saldoTotalCentavos()` no cambia tras un `transferir()` exitoso.

El observador y el registro son los puntos de enganche con replicacion y durabilidad. `setObservador()` instala un `Consumer<Transaccion>` (lo pone el lider) al que se notifica cada commit para propagar la transaccion a las replicas y al log durable; `setConteoStorage()` inyecta el numero de transacciones presentes en Cloud Storage, consultable con `txEnStorage()`.

Listing 33: Puntos de enganche en `CuentaRepository.java`

```
public void setObservador(Consumer<Transaccion> o) { this.observador = o; }
public void setConteoStorage(LongSupplier s) { this.conteoStorage = s; }
public long txEnStorage() { return conteoStorage.getAsLong(); }
```

La replica aplica los cambios sin revalidar: `aplicarReplica()` confía en que el líder ya validó el saldo, por lo que solo mueve el monto, registra la tx y avanza la secuencia local al máximo entre la actual y `t.seq()` mediante `accumulateAndGet(t.seq(), Math::max)`. Reusa el mismo bloqueo ordenado por id para no romper la coherencia frente a transferencias locales concurrentes.

Listing 34: Aplicación idempotente en la replica, `CuentaRepository.java`

```
origen.setSaldoCentavos(origen.getSaldoCentavos() - t.montoCentavos());
destino.setSaldoCentavos(destino.getSaldoCentavos() + t.montoCentavos());
registro.agregar(t);
totalTransferencias.increment();
ultimaTxId = t.seq();
secuencia.accumulateAndGet(t.seq(), Math::max);
```

4.4 Errores de negocio y mapeo a códigos REST

`transferir()` no lanza excepciones de negocio: devuelve un `Resultado` con un `Estado` de la enumeración. La capa HTTP, `server/TransferHandler.java`, traduce ese estado a un código de respuesta y un cuerpo JSON. El éxito devuelve el seq asignado; los fallos de negocio se agrupan en 400 salvo la cuenta inexistente, que es 404.

Estado	HTTP	Significado
OK	200	Commit; el cuerpo incluye el seq.
NO_ENCONTRADA	404	Origen o destino no existe.
MISMA_CUENTA	400	Origen y destino iguales.
MONTO_INVALIDO	400	Monto menor o igual a cero.
SALDO_INSUFICIENTE	400	El origen no cubre el monto.

Listing 35: Mapeo de Estado a respuesta REST en `TransferHandler.java`

```
CuentaRepository.Resultado r = repo.transferir(origen, destino, centavos);
switch (r.estado) {
    case OK: return Respuesta.json(200, "{\"status\":\"ok\",\"seq\":\"" +
        ↪ r.seq + "\"}");
    case NO_ENCONTRADA: return Respuesta.error(404, "Cuenta no encontrada");
    case MISMA_CUENTA: return Respuesta.error(400, "Origen y destino iguales");
    case MONTO_INVALIDO: return Respuesta.error(400, "Monto invalido");
    case SALDO_INSUFICIENTE: return Respuesta.error(400, "Saldo insuficiente");
    default: return Respuesta.error(500, "Error interno");
}
```

IMPORTANTE

La validación de saldo ocurre dentro de los candados, no antes: comprobar el saldo fuera de la sección crítica abriría una condición de carrera (dos retiros concurrentes podrían pasar la verificación y dejar saldo negativo).

5 Replicación y tolerancia a fallos

El banco replica su estado desde un nodo *líder* hacia uno o varios nodos *replica* mediante un log de transacciones ordenado por número de secuencia y un transporte TCP directo líder a replica (no se usa Pub/Sub). El líder acepta conexiones de replica, las pone al día (*catch-up*) desde la secuencia que cada una reporta y después les empuja en vivo cada transacción nueva. Si una replica se cae y vuelve, reanuda exactamente donde se quedó: si procesó hasta la secuencia 4500, al reconectar pide y recibe la 4501 en adelante. Así, apagar y encender replicas no pierde ni duplica transacciones.

La reanudación por secuencia sobrevive incluso a *matar el proceso* de la replica (no solo a perder la conexión), gracias a un punto de control local en disco (`com/escom/banco/replicacion/PuntoControl.java`) que persiste el par `{secuencia + saldos}`. Al revivir, la replica carga ese checkpoint y manda `RESUME <seq>` desde donde se quedó, en vez de borrar todo y descargar el estado completo. El único caso en que su checkpoint puede quedar por encima del líder (una caída total donde el líder se recuperó del log GCS rezagado) lo resuelve el líder ordenando `RESET`, tras lo cual la replica recarga el CSV y reanuda con `RESUME 0` hasta converger.

5.1 Log por número de secuencia

La pieza central es `com/escom/banco/data/RegistroTransacciones.java`, un log de transacciones indexado por su número de secuencia. Cada `com/escom/banco/model/Transaccion.java` es un record con `seq`, `origenId`, `destinoId` y `montoCentavos`; el campo `seq` fija un orden total. El log se respalda en un `ConcurrentSkipListMap`, que mantiene las entradas ordenadas por clave aunque varios hilos agreguen transacciones de forma concurrente.

Listing 36: RegistroTransacciones: append y lectura ordenada

```
private final ConcurrentSkipListMap<Long, Transaccion> log = new
    ↪ ConcurrentSkipListMap<>();

public void agregar(Transaccion t) {
    log.put(t.seq(), t);
}

/** Transacciones con seq estrictamente mayor a la dada, en orden de seq. */
public List<Transaccion> desde(long seq) {
    return new ArrayList<>(log.tailMap(seq, false).values());
}
```

El método `desde(N)` devuelve todas las transacciones con secuencia estrictamente mayor a `N`, ya ordenadas. Esta operación es la base tanto del *catch-up* de una replica que se reincor-

pora como del streaming en vivo. Para que el stream sea contiguo y sin huecos, el orden de insercion en el registro debe coincidir con el orden de los `seq`: eso lo asegura el lider, que en `CuentaRepository.transferir` serializa bajo un `ReentrantLock` interno (`seqLock`) el tramo que incrementa la secuencia, agrega la transaccion al registro y avisa al observador. Asi una transaccion con `seq` mayor nunca aparece en el log antes que una con `seq` menor, aunque varios hilos transfieran a la vez.

5.2 El lider: catch-up y streaming TCP

El lado lider es `com/escom/banco/replicacion/ServidorReplicacion.java`. Abre un `ServerSocket` en un hilo *daemon* de aceptacion y, por cada replica que se conecta, lanza un hilo que la atiende. El protocolo de saludo es una sola linea `RESUME N`: la replica declara la ultima secuencia que ya tiene aplicada, y a partir de ahi el lider le envia todo lo que falta y luego lo nuevo, sirviendose siempre de `repo.registro().desde(enviado)`.

Listing 37: ServidorReplicacion: RESET si la replica esta adelantada, catch-up y streaming

```
long enviado = leerResume(in.readLine()); // "RESUME N"
// Si la replica pide reanudar MAS ADELANTE del estado actual del lider,
// su checkpoint quedo adelantado: es el caso de caida TOTAL, donde el
// lider se recupero del log GCS (rezagado) y "retrocedio" a una seq
// menor. Su estado seria inconsistente -> ordenarle reconstruir la base.
if (enviado > repo.secuenciaActual()) {
    out.write("RESET\n");
    out.flush();
    return;
}
while (activo && !sock.isClosed()) {
    List<Transaccion> nuevas = repo.registro().desde(enviado);
    if (nuevas.isEmpty()) {
        Thread.sleep(1);
        continue;
    }
    for (Transaccion tx : nuevas) {
        out.write(CodecTransaccion.aLinea(tx));
        out.write("\n");
        enviado = tx.seq();
    }
    out.flush();
}
```

Tras el catch-up (todo lo posterior a `N`), el mismo bucle convierte el envio en streaming continuo: cuando no hay nada nuevo duerme un milisegundo y reintenta; cuando aparecen transacciones recién agregadas al log, las empuja en orden y avanza el cursor `enviado` a la ultima `seq` servida. El parseo del saludo lo hace `leerResume`, que ante una linea ausente o mal formada degrada a `0` (la replica recibe el log completo).

NOTA

El líder no recuerda por replica cuanto le envío: el cursor enviado vive en el hilo de esa conexión y se reconstruye en cada reconexión a partir del RESUME N que manda la replica. El estado de avance lo posee la replica.

Antes de empezar el catch-up, el líder compara la secuencia pedida contra la suya: si enviado es estrictamente mayor que `repo.secuenciaActual()`, la replica trae un checkpoint mas nuevo que el estado del propio líder. Esto solo ocurre tras una caída total de los tres nodos, donde el líder se reconstruye a partir del log durable de Cloud Storage (rezagado) y volvió en una secuencia menor. En ese caso el líder no puede servir el delta pedido sin dejar a la replica divergente, así que responde RESET y cierra; la reconciliación la hace la replica (ver la subsección del punto de control).

5.3 La replica: aplicar y reconectar

El lado replica es `com/escom/banco/replicacion/ClienteReplicacion.java`. Corre en su propio hilo daemon: se conecta al líder, imprime en su log el renglón que el profesor pide ver al levantarla, envía RESUME con `repo.secuenciaActual()` y luego lee línea por línea. Cada transacción se aplica con `repo.aplicarReplica(...)` (o vía `puntoControl.aplicar`, si la replica de producción tiene checkpoint). Si la conexión se cae, el bucle externo reintenta tras una breve espera y reanuda desde la secuencia actual de la replica.

Listing 38: ClienteReplicacion: log de reanudación, RESUME, RESET y aplicar en vivo

```
long seq = repo.secuenciaActual();
// El profesor pide ver este renglón en el log de la replica al
// levantarla: confirma que reanuda por secuencia (no borra+descarga).
System.out.println("Me quede en la secuencia " + seq
    + ", enviame desde " + (seq + 1));
out.write("RESUME " + seq + "\n");
out.flush();

PuntoControl pc = this.puntoControl;
String linea;
while (activo && (linea = in.readLine()) != null) {
    if (linea.equals("RESET")) {
        // checkpoint ADELANTE del estado durable del líder: recargar
        // la base y reanudar desde 0 para no quedar divergente.
        Runnable r = alResetear;
        if (r != null) r.run();
        break;
    }
    try {
        Transaccion tx = CodecTransaccion.deLinea(linea);
        if (pc != null) pc.aplicar(repo, tx);
        else repo.aplicarReplica(tx);
    } catch (RuntimeException ex) {
        // Línea mal formada: no debe matar el hilo; se reconecta con RESUME.
        break;
    }
}
```

```
}
```

El renglón `Me quede en la secuencia X`, enviame desde `X+1` se imprime en cada (re)conexion y es la evidencia de que la replica reanuda por secuencia en lugar de borrar y descargar el estado completo. La serializacion en el cable es responsabilidad de `com/escom/banco/replicacion/CodecTransaccion.java` que codifica cada transaccion como una línea ASCII de cuatro campos separados por comas (`seq, origenId, destinoId, montoCentavos`) con `aLinea` y la reconstruye con `deLinea`. Al aplicar, `aplicarReplica` mueve el monto entre cuentas, agrega la transaccion al log local y avanza la secuencia a `t.seq()`, de modo que el siguiente `RESUME` refleje el avance. Una línea mal formada (p. ej. un corte de conexion a media línea) rompe el bucle pero no mata el hilo: se reconecta y vuelve a mandar `RESUME`.

5.4 Punto de control local y reconciliacion por RESET

El catch-up TCP basta cuando la replica solo perdio la conexion, porque su estado en memoria sigue intacto. Pero si se *mata el proceso*, la memoria se pierde: sin nada mas, la replica tendria que arrancar del CSV (secuencia 0) y descargar todo otra vez. El punto de control local (`com/escom/banco/replicacion/PuntoControl.java`) evita ese borron: persiste en disco el par `{secuencia + saldos}` (un archivo binario plano, no una base de datos) para que tras revivir el proceso la replica reanude por secuencia. El log durable para la caida total de los tres nodos sigue siendo Cloud Storage, solo en el lider.

El snapshot se toma periodicamente (cada `BANCO_CHECKPOINT_SEG` segundos, por omision 3) y, ademas, de forma exacta al recibir `SIGTERM` mediante un *shutdown hook* que llama a `guardarFinal`. La copia en memoria se hace bajo el mismo candado con que se aplican las transacciones, asi el par `{secuencia, saldos}` siempre es coherente; el volcado a disco es atomico (archivo temporal + `rename`) y los volcados se serializan entre el hilo timer y el shutdown hook.

Listing 39: PuntoControl: cargar parsea TODO antes de mutar el repo

```
public long cargar(CuentaRepository repo) {
    if (!Files.exists(archivo)) return -1;
    long seq;
    int[] ids;
    long[] saldos;
    // Se parsea TODO el archivo a buffers temporales ANTES de tocar el repo:
    // un cuerpo truncado revienta aqui sin dejar el repo mitad-CSV/mitad-checkpoint.
    try (DataInputStream in = /* ...BufferedInputStream... */) {
        if (in.readInt() != MAGIA) { /* ... */ return -1; }
        seq = in.readLong();
        int count = in.readInt();
        // ... validar count y leer ids[]/saldos[] ...
    } catch (IOException | RuntimeException e) {
        return -1; // checkpoint malo: se arranca desde el CSV
    }
    for (int i = 0; i < ids.length; i++) {
        Cuenta c = repo.get(ids[i]);
        if (c != null) c.setSaldoCentavos(saldos[i]);
    }
    repo.restaurarSecuencia(seq);
}
```



```
    return seq;
}
```

cargar no lanza nunca: si el archivo falta, tiene formato invalido o el cuerpo quedo truncado, devuelve -1 y la replica arranca del CSV con RESUME 0. Como parsea el archivo entero a buffers antes de tocar el repo, un checkpoint a medias deja el repo intacto en su base CSV en vez de mezclar saldos. Al restaurar, `repo.restaurarSecuencia(seq)` fija la secuencia, la ultima tx y el total de transferencias (igual a `seq`, sin huecos).

La otra mitad de la reconciliacion es el RESET. Cuando el lider responde RESET (porque la replica pidio reanudar por encima de su secuencia tras una caida total y recuperacion desde GCS), la replica ejecuta el callback `alResetear`: recarga la base desde el CSV y vuelve la secuencia a 0, de modo que la siguiente vuelta del bucle se reconecta con RESUME 0 y el lider la pone al dia desde cero. Asi la replica converge al estado del lider en lugar de quedar adelantada.

5.5 Rol por entorno y resincronizacion ante fallos

El rol de cada nodo se decide por variables de entorno en `com/escom/banco/server/MainServer.java`: sin `BANCO_LIDER_HOST` el nodo es LIDER y levanta el `ServidorReplicacion`; CON `BANCO_LIDER_HOST` es REPLICA y arranca el `ClienteReplicacion` apuntando a ese host. El puerto de replicacion sale de `BANCO_REPLICACION_PUERTO` (por omision 9090). En la replica, `MainServer` carga su punto de control (`cargarPuntoControl`) antes de servir trafico, cablea el callback `alResetear` (recargar el CSV) y registra el shutdown hook que guarda el snapshot exacto en SIGTERM.

Listing 40: MainServer: rol por entorno, RESET y shutdown hook del checkpoint

```
if (esLider()) {
    new ServidorReplicacion(CuentaRepository.get(), puertoRepl).iniciar();
    System.out.println("Rol: LIDER - replicacion TCP en puerto " + puertoRepl);
} else {
    String liderHost = System.getenv("BANCO_LIDER_HOST");
    CuentaRepository repo = CuentaRepository.get();
    ClienteReplicacion cliente = new ClienteReplicacion(repo, liderHost, puertoRepl);
    // Si el lider ordena RESET: recargar la base del CSV y reanudar con RESUME 0.
    cliente.setAlResetear(() -> {
        try { AlumnosLoader.cargar(repo, csvPath); } catch (Exception e) { /* ... */ }
        repo.restaurarSecuencia(0);
    });
    if (puntoControl != null) {
        cliente.setPuntoControl(puntoControl);
        puntoControl.iniciar(repo);
        // Snapshot exacto en un apagado ordenado (SIGTERM): "detiene el proceso".
        Runtime.getRuntime().addShutdownHook(
            new Thread(() -> puntoControl.guardarFinal(repo),
                ↪ "punto-control-final"));
    }
    cliente.iniciar();
    System.out.println("Rol: REPLICA - siguiendo a " + liderHost + ":" + puertoRepl);
}
```

El checkpoint se ubica con `BANCO_CHECKPOINT` (ruta absoluta que debe persistir entre reinicios del proceso) y su periodo con `BANCO_CHECKPOINT_SEG`.

La tolerancia a fallos emerge de combinar las tres piezas anteriores. Si se apaga una replica, el lider solo ve cerrarse su socket y el hilo que la atendia termina; el resto del sistema sigue. Cuando esa replica se enciende de nuevo, reconecta, manda `RESUME 4500` y el lider responde con la secuencia 4501 en adelante (catch-up) antes de retomar el streaming en vivo. Como el avance se deriva siempre del numero de secuencia, no hay huecos ni duplicados.

IMPORTANTE

Una replica nunca origina transacciones propias: `aplicarReplica` solo aplica, en el orden recibido, transacciones ya validadas por el lider. El unico caso en que su secuencia puede quedar por encima de la del lider es una caida total seguida de una recuperacion del lider desde el log GCS rezagado; ese caso ya no rompe la consistencia, porque el lider responde `RESET` y la replica recarga el CSV y reanuda con `RESUME 0` hasta converger.

6 Persistencia y recuperacion en Cloud Storage

El banco mantiene su estado vivo en memoria y en un CSV local, pero ambos desaparecen si caen todos los nodos. Para sobrevivir a ese escenario, el lider escribe un *log durable* de transacciones en un bucket de Google Cloud Storage (GCS): cada transferencia confirmada se persiste como un objeto JSON independiente. El componente `Almacen/AlmacenGcs.java` coordina la escritura asincrona y la recuperacion, mientras que `Almacen/GcsSubidor.java` hace el dialogo real con la API de GCS usando solo el JDK. La activacion ocurre en `server/MainServer.java`, que solo arranca el log en el lider y solo si esta definida la variable de entorno `BANCO_GCS_BUCKET`.

NOTA

El sistema tiene dos recuperaciones con alcances distintos. El log de Cloud Storage descrito en este capitulo cubre la *caida total* de los tres nodos: cuando se pierde el estado en memoria de todos, el lider reconstruye el estado completo reaplicando el log desde GCS. El *punto de control local* de cada replica (`replicacion/PuntoControl.java`, configurado con `BANCO_CHECKPOINT`; detallado en el capitulo de replicacion) cubre el caso opuesto: una replica que se reinicia mientras el lider sigue vivo reanuda desde su ultima secuencia sin redescargar todo. Mecanismos complementarios, no sustitutos.

6.1 Escritura asincrona que no bloquea la transferencia

La regla de diseno es que el camino critico de una transferencia nunca debe esperar a la red. Por eso `AlmacenGcs` usa una cola (`BlockingQueue`) y un unico hilo escritor: cuando el repositorio confirma una transaccion, solo la encola con `registrar()` y sigue; el lider permanece como fuente de verdad y el objeto en GCS va detras.

Listing 41: El registro encola y no bloquea; un hilo daemon vacia la cola

```
/** Encola una transaccion para persistirla (no bloquea). */
public void registrar(Transaccion t) {
```

```

    if (activo) cola.offer(t);
}

public void iniciar(long escritosIniciales) {
    escritos.set(Math.max(0, escritosIniciales));
    activo = true;
    worker = new Thread(this::correr, "gcs-log");
    worker.setDaemon(true);
    worker.start();
}

```

El hilo `correr()` hace `poll` con `timeout` para drenar la cola incluso después de `detener()`, y cada subida pasa por `subirConReintentos()`, que prueba hasta `REINTENTOS` (3) veces con una espera creciente. Si la subida tiene éxito, incrementa el contador `escritos`; si falla tras los reintentos, deja constancia en `stderr` pero no detiene el log.

Listing 42: Reintentos con respiro creciente ante fallos transitorios de red

```

private boolean subirConReintentos(Transaccion t) {
    for (int i = 0; i < REINTENTOS; i++) {
        try {
            if (subidor.subir(t)) return true;
        } catch (Exception e) {
            // reintentamos con un respiro
        }
        try { Thread.sleep(100L * (i + 1)); }
        catch (InterruptedException e) { return false; }
    }
    return false;
}

```

NOTA

La subida real se delega en la interfaz `AlmacenGcs.Subidor`. Esto desacopla la lógica de cola y reintentos de la red, de modo que las pruebas pueden inyectar un subidor falso y verificar el comportamiento sin tocar GCS.

6.2 El subidor real: solo JDK contra la API JSON de GCS

`GcsSubidor` implementa `AlmacenGcs.Subidor` sin la librería cliente de Google: usa el `HttpClient` del JDK y la API JSON de GCS. Como GCS no permite *append*, se escribe un objeto por transacción en la ruta `transactions/{seq}.json`, lo que además da un nombre estable e idempotente por número de secuencia.

Listing 43: Un objeto por transacción via `uploadType=media`

```

@Override
public boolean subir(Transaccion t) throws Exception {
    String nombre = URLEncoder.encode("transactions/" + t.seq() + ".json",

```

```

↪ StandardCharsets.UTF_8);
String url = "https://storage.googleapis.com/upload/storage/v1/b/" + bucket
    + "/o?uploadType=media&name=" + nombre;
String cuerpo = Json.toJson(t);
HttpRequest req = HttpRequest.newBuilder(URI.create(url))
    .header("Authorization", "Bearer " + obtenerToken())
    .header("Content-Type", "application/json")
    .timeout(Duration.ofSeconds(10))
    .POST(HttpRequest.BodyPublishers.ofString(cuerpo)).build();
int sc = http.send(req, BodyHandlers.discarding()).statusCode();
return sc ≥ 200 && sc < 300;
}

```

La autenticación no usa claves de servicio en disco: el método `obtenerToken()` pide un token OAuth al *metadata server* de la VM, que solo responde si la petición lleva la cabecera `Metadata-Flavor: Google`. El token se cachea y se renueva 60 segundos antes de su expiración para no pedir uno por cada subida.

Listing 44: Token OAuth del metadata server, cacheado y renovado antes de expirar

```

private String obtenerToken() throws Exception {
    long ahora = System.currentTimeMillis();
    if (token ≠ null && ahora < tokenVenceMs) return token;
    HttpRequest req = HttpRequest.newBuilder(URI.create(META_TOKEN))
        .header("Metadata-Flavor", "Google")
        .timeout(Duration.ofSeconds(5)).GET().build();
    JsonObject j = Json.parse(http.send(req, BodyHandlers.ofString()).body());
    token = j.get("access_token").getString();
    long venceEnS = j.has("expires_in") ? j.get("expires_in").getLong() : 3600;
    tokenVenceMs = ahora + (venceEnS - 60) * 1000L; // renovamos 60s antes
    return token;
}

```

El mismo subidor sabe listar y bajar objetos. `contarExistentes()` recorre el bucket con paginación (prefix=transactions/ y nextPageToken) y devuelve cuantos objetos hay; `leerTodas()` lista y luego descarga cada objeto, reintentando unas pocas veces ante fallos transitorios y propagando el error si los agota (una descarga perdida corrompería el log al reaplicarse a medias). Parsea a mano los campos simples (seq, origenId, destinoId, montoCentavos) para reconstruir cada Transacción.

6.3 Siembra del contador y recuperación en frío

Al arrancar, el líder siembra el contador de transacciones persistidas con el número que devuelve `recuperar()` (las que ya estaban en el bucket), pasándolo a `iniciar(recuperadas)`, de modo que la métrica continúe donde se quedó y no reinicie en cero tras un reinicio. El propio `Almacen6cs` ofrece además un `iniciar()` sin argumentos que cuenta los objetos con `contarExistentes()`, pero el líder no usa ese camino: aprovecha el conteo que ya obtuvo la recuperación.

La recuperación en frío es el escenario en que fallaron todos los nodos y se pierde el estado en memoria. Antes de servir tráfico, el líder reaplica el log completo sobre el repositorio mediante

`recuperar()`: lee todas las transacciones con `leerTodas()`, las ordena por `seq` y las reaplica en orden con la funcion recibida.

Listing 45: Reaplicacion ordenada del log; falla ruidosa si no se puede leer

```
public long recuperar(Consumer<Transaccion> aplicar) {
    List<Transaccion> txs;
    try {
        txs = new ArrayList<>(subidor.leerTodas());
    } catch (Exception e) {
        // No degradamos a estado base en silencio: servir el CSV con
        // secuencia=0 haria que las nuevas tx reutilicen seqs ya usados y
        // corrompan el log durable. Fallamos ruidosamente para que el
        // arranque aborte y el nodo no sirva un estado falso.
        throw new IllegalStateException(
            "GCS: no se pudo leer el log para recuperar; se aborta el "
            + "arranque para no servir estado base corrupto", e);
    }
    txs.sort(Comparator.comparingLong(Transaccion::seq));
    for (Transaccion t : txs) aplicar.accept(t);
    return txs.size();
}
```

El cableado vive en `MainServer.iniciarAlmacenGcs()`: primero reaplica el log (`repo::aplicarReplica`), luego arranca el worker con el numero de transacciones recuperadas, engancha el observador para los nuevos commits y, por ultimo, registra un *shutdown hook* que drena la cola en un apagado ordenado (SIGTERM), pues una tx ya ackeada con HTTP 200 puede seguir pendiente de subir y se perderia del log si el JVM termina sin vaciar la cola. La funcion `aplicarReplica` reaplica sin revalidar saldos ni reescribir en GCS, de modo que la recuperacion no genera un ciclo de subidas.

Listing 46: Recuperacion antes de servir, observador y shutdown hook (MainServer)

```
long recuperadas = almacen.recuperar(repo::aplicarReplica);
// ...
almacen.iniciar(recuperadas);
repo.setObservador(almacen::registrar);
repo.setConteoStorage(almacen::escritos);
// Drena la cola asincrona en un apagado ordenado (SIGTERM): una tx ya
// ackeada (HTTP 200) puede seguir pendiente de subir a GCS y se perderia
// del log durable si el JVM termina sin vaciar la cola.
Runtime.getRuntime().addShutdownHook(new Thread(() -> almacen.detener()));
```

IMPORTANTE

La recuperacion reaplica sobre el estado de las cuentas del CSV. Las cuentas creadas via `/api/register` que no esten en el CSV no se recuperan; la prueba del proyecto usa las cuentas del dataset.

6.4 Exposición de la métrica en el dashboard

El conteo de transacciones persistidas se publica para el panel a través del repositorio. `MainServer` conecta `repo.setConteoStorage(almacen::escritos)`, y `data/CuentaRepository.java` expone el valor con `txEnStorage()`, que devuelve `-1` cuando el log GCS está deshabilitado. `server/StatsHandler.java` lo incluye en el JSON de estadísticas bajo la clave `txEnStorage`, de manera que el dashboard muestra en tiempo real cuantas transacciones ya quedaron durables en Cloud Storage frente a las que aun están pendientes en la cola.

7 Generador de carga y rendimiento

El generador de carga vive en `src/main/java/com/escom/banco/carga/GeneradorCarga.java`. Es una herramienta de línea de comandos en Java puro, sin dependencias nuevas, que golpea al nodo líder durante un minuto con una mezcla de 80 % lecturas y 20 % transferencias concurrentes, y al terminar comprueba que la suma total de saldos no haya cambiado (invariante de conservación del dinero).

7.1 Modelo de carga

La herramienta primero se autentica como el usuario `carga`, lee el número de cuentas y el saldo total inicial desde `/stats`, y exige al menos dos cuentas para poder transferir. Cada petición se decide al azar: con probabilidad del 80 % es una lectura `GET /api/accounts/{id}` y con el 20 % restante es una transferencia `POST /api/transactions/transfer` entre dos cuentas elegidas al azar. Los montos se generan en centavos y se serializan con `BigDecimal` para no perder exactitud al pasarlos a JSON.

Listing 47: Decision 80/20 por petición (lanzarUna)

```
boolean lectura = rnd.nextInt(100) < 80;
if (lectura) {
    int id = 1 + rnd.nextInt(cuentas);
    req = HttpRequest.newBuilder(URI.create(base + "/api/accounts/" + id))
        .header("Authorization", autorizacion).GET().build();
} else {
    int origen = 1 + rnd.nextInt(cuentas);
    int destino = 1 + rnd.nextInt(cuentas);
    long montoCent = 1 + rnd.nextLong(cfg.montoMaxCent());
    // ...
}
```

La parametrización de una corrida se concentra en el record `Config`, que distingue las peticiones en vuelo del número de clientes HTTP independientes.

Listing 48: Parametros de una corrida (Config)

```
public record Config(String host, int puerto, int segundos, int enVuelo,
    long montoMaxCent, int clientes) {
    /** Conveniencia: un solo HttpClient (comportamiento original). */
    public Config(String host, int puerto, int segundos, int enVuelo, long
```

```

    ↪ montoMaxCent) {
        this(host, puerto, segundos, enVuelo, montoMaxCent, 1);
    }
}

```

7.2 Motor asincrono con ventana en vuelo

En lugar de dedicar un hilo del sistema operativo por petición (un envío bloqueante a la vez), el generador mantiene una ventana de N peticiones EN VUELO usando `HttpClient.sendAsync()` y un `Semaphore` que limita cuantas peticiones pueden estar pendientes a la vez. Cada permiso se adquiere antes de lanzar la petición y se libera en el callback `whenComplete`, de modo que una sola maquina sostiene mucha mas concurrencia sin gastar N hilos.

Listing 49: Bucle del motor con semaforo (motor)

```

private void motor(HttpClient cliente, int cuentas, int ventana, long fin) {
    Semaphore permisos = new Semaphore(ventana);
    try {
        while (System.nanoTime() < fin) {
            permisos.acquire();
            if (System.nanoTime() ≥ fin) { permisos.release(); break; }
            lanzarUna(cliente, cuentas, permisos);
        }
        permisos.acquire(ventana); // drenar las peticiones en vuelo de este motor
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}

```

Al terminar la fase de envío, el motor adquiere la ventana completa (`permisos.acquire(ventana)`) para drenar las peticiones que aun siguen en vuelo antes de devolver el control. El conteo de exitos y rechazos se acumula con `LongAdder`, una estructura concurrente que escala mejor que un contador unico cuando muchos callbacks incrementan a la vez.

7.3 Parametro clientes (K) y verificacion del invariante

El parametro `clientes` (K) crea K instancias de `HttpClient` independientes, cada una con su propio hilo selector de I/O. La ventana total de peticiones en vuelo se reparte entre ellas (`ventana = enVuelo / k`), de modo que UNA sola maquina generadora no se tope con el limite de un unico selector. El `HttpClient` del campo compartido se usa solo para autenticarse y leer `/stats`; la carga real la llevan los K clientes creados por hilo.

Listing 50: Reparto de la ventana entre K clientes (ejecutar)

```

int k = Math.max(1, cfg.clientes());
int ventana = Math.max(1, cfg.enVuelo() / k); // ventana POR cliente
long fin = System.nanoTime() + cfg.segundos() * 1_000_000_000L;

Thread[] hilos = new Thread[k];

```

```

for (int i = 0; i < k; i++) {
    HttpClient cliente = HttpClient.newBuilder()
        .version(HttpClient.Version.HTTP_1_1)
        .connectTimeout(Duration.ofSeconds(5)).build();
    hilos[i] = new Thread(() -> motor(cliente, cuentas, ventana, fin), "motor-" + i);
}
for (Thread h : hilos) h.start();
for (Thread h : hilos) h.join();

```

Una vez que todos los hilos terminan, se vuelve a leer el saldo total y se compara con el inicial. Si coinciden, el dinero se conservo exacto y se imprimen las transacciones y lecturas exitosas; si difieren, se reporta la inconsistencia indicando el delta en centavos y se sale con código de error.

Listing 51: Verificación del invariante (main)

```

if (r.invarianteOk()) {
    System.out.println("INVARIANTE OK: la suma total se conservo exacta.");
} else {
    System.out.printf("INCONSISTENCIA: el total cambio en %s (centavos: %d).%n",
        centavosAJson(r.deltaCent()), r.deltaCent());
    System.exit(1);
}

```

La interfaz de línea de comandos es `host puerto [segundos] [enVuelo] [clientes]`, con valores por omisión de 60 segundos, 512 peticiones en vuelo y 1 cliente:

Listing 52: Uso por línea de comandos

```

Uso: GeneradorCarga <host> <puerto> [segundos=60] [enVuelo=512] [clientes=1]

```

7.4 Resultados

La progresión de rendimiento medida (banco fijo en una `e2-standard-4`) muestra como se fue moviendo el cuello de botella. La primera versión sobre `com.sun.net.httpserver` topaba en unas 16,000 ops/s por el dispatcher de un solo hilo. Al pasar a un servidor NIO propio se alcanzaron 25,000 a 28,000 ops/s con un solo generador; con NIO y un generador `c3-standard-22` usando 1 `HttpClient` se llegó a unas 60,000 ops/s; y con `c3-standard-22` y `K` igual a 2 o 3 `HttpClient` se saturó el banco en unas 110,000 ops/s. Sobre ese punto saturado, dos optimizaciones del lado del banco subieron el techo: el lote de *allocations* (precomputar campos y armar el JSON sin `BigDecimal` ni `split`) aportó cerca de un 7 %, y cambiar el recolector de basura a `-XX:+UseParallelGC` con heap fijo de 8 GB y `AlwaysPreTouch` aportó cerca de un 10 %, llegando a un pico de unas 128,000 ops/s con la CPU del banco al 96 a 98 %. A partir de ahí el banco es CPU-bound en la `e2-standard-4` (techo físico de la máquina), no por el generador: el generador óptimo es una `c3-standard-22` con `K` igual a 3.

Los números oficiales por escenario se midieron en 60 s, con un generador `c3-standard-22` y `K` igual a 3, en la configuración de concurso con Cloud Storage y replicación activos. El invariante

de conservacion se mantuvo exacto en los tres escenarios.

Escenario	Lecturas	Transferencias	Ops/s aprox.
Nodos 1+2+3	5,523,490	1,379,928	115,057
Nodos 1+2	6,231,510	1,558,375	129,831
Nodo 1	6,353,653	1,588,544	132,370

El sobre costo (overhead) de replicacion es de aproximadamente 13 % con dos replicas y de aproximadamente 2 % con una sola replica. El puntaje del concurso pondera las transferencias 4x sobre las lecturas y suma los tres escenarios.

8 Dashboard de observabilidad

El sistema expone un panel web de una sola pagina que muestra, en tiempo real, el estado de cada nodo del cluster: rol, numero de cuentas, saldo total, numero de transferencias, identificador de la ultima transaccion, porcentajes de CPU, RAM y disco, y el numero de transacciones registradas en Cloud Storage. El panel no necesita servidor adicional ni dependencias de frontend: es HTML estatico servido por el propio nodo y alimentado por dos endpoints JSON.

8.1 Pagina servida en GET /

El recurso `src/main/resources/dashboard.html` se empaqueta dentro del jar y lo entrega `PaginaHandler` en `GET /`. Para evitar leer el archivo en cada peticion, el manejador lo carga una sola vez en un arreglo de bytes estatico y lo cachea; si el recurso falta responde 500 y a cualquier ruta distinta de la raiz responde 404.

Listing 53: `PaginaHandler.java`: dashboard cacheado en bytes

```
private static final byte[] HTML = cargar();

@Override
public Respuesta manejar(Solicitud s) {
    if (!"GET".equalsIgnoreCase(s.metodo())) return Respuesta.sinCuerpo(405);
    if (!"/".equals(s.path())) return Respuesta.sinCuerpo(404);
    if (HTML == null) return Respuesta.sinCuerpo(500);
    return Respuesta.html(200, HTML);
}
```

8.2 Estado de un nodo: GET /stats

`StatsHandler` responde en `GET /stats` con el snapshot del nodo local. Es publico (sin JWT) porque solo expone monitoreo de un nodo. El metodo estatico `StatsHandler.snapshot()` construye un mapa ordenado que tambien reutiliza el panel agregado. Determina el rol leyendo la variable de entorno `BANCO_LIDER_HOST` (vacía implica `LIDER`, definida implica `REPLICA`) y publica el saldo total en centavos como `long` exacto para verificar el invariante de consistencia en una sola peticion.

Listing 54: StatsHandler.java: snapshot del nodo

```

out.put("rol", rol);
out.put("cuentas", repo.tamano());
out.put("saldoTotalCentavos", totalCent);
out.put("transferencias", repo.totalTransferencias());
out.put("ultimaTxId", repo.ultimaTxId());
out.put("secuencia", repo.secuenciaActual());
out.put("cpu", SistemaMetricas.cpuPorcentaje());
out.put("ram", SistemaMetricas.ramPorcentaje());
out.put("disco", SistemaMetricas.discoPorcentaje());
out.put("txEnStorage", repo.txEnStorage()); // -1 si el log GCS esta deshabilitado

```

Las metricas de sistema viven en `SistemaMetricas` y se calculan solo con APIs del JDK, sin dependencias externas: el %CPU y el %RAM se obtienen del `OperatingSystemMXBean` de `com.sun.management`, y el %Disco se mide con un `File` sobre la raiz del sistema de archivos.

Listing 55: SistemaMetricas.java: CPU y RAM sin dependencias

```

public static double cpuPorcentaje() {
    double carga = OS.getCpuLoad();
    return carga < 0 ? 0.0 : redondear(carga * 100.0);
}

public static double ramPorcentaje() {
    long total = OS.getTotalMemorySize();
    if (total ≤ 0) return 0.0;
    long usada = total - OS.getFreeMemorySize();
    return redondear(100.0 * usada / total);
}

```

8.3 Vista agregada del cluster: GET /panel

`PanelHandler` responde en `GET /panel` y construye la vista completa del cluster que consume el dashboard. El lider toma su propio `StatsHandler.snapshot()` y, ademas, consulta el `/stats` de cada peer. La consulta es un `fetch` del lado del servidor (con `HttpClient`), lo que evita problemas de CORS en el navegador y centraliza la agregacion en un solo origen. Cada nodo se envuelve con metadatos `host` y `alcanzable`; si un peer no responde dentro del timeout de 2 s, se marca `alcanzable:false` sin abortar el resto del panel.

Listing 56: PanelHandler.java: agregado lider + peers

```

List<Map<String, Object>> nodos = new ArrayList<>();
nodos.add(conMeta(idLocal, true, StatsHandler.snapshot()));
for (String peer : peers) nodos.add(consultarPeer(peer));
// ...
private Map<String, Object> consultarPeer(String hostPuerto) {
    try {
        HttpRequest req = HttpRequest.newBuilder(URI.create("http://" + hostPuerto +
            ↪ "/stats"))

```

```

        .timeout(Duration.ofSeconds(2)).GET().build();
        String cuerpo = http.send(req, BodyHandlers.ofString()).body();
        // ...
        return conMeta(hostPuerto, true, m);
    } catch (Exception e) {
        return conMeta(hostPuerto, false, new LinkedHashMap<>());
    }
}

```

NOTA

Como `HttpClient.send` hacia los peers es bloqueante (hasta 2 s), `PanelHandler` se asigna al pool `WORKER`; si corriera en un reactor lo estancaría. Así un peer caído nunca cuelga el panel del resto del cluster.

8.4 Refresco reactivo en el navegador

El JavaScript embebido en `dashboard.html` consulta `/panel`, dibuja una tarjeta por nodo (con badge `LIDER`, `REPLICA` o `CAIDO` según el campo `alcanzable`) y compara los `saldoTotalCentavos` de los nodos vivos para señalar si los saldos son consistentes entre nodos. El refresco es reactivo: tras la carga inicial, un `setInterval` repite la consulta cada 2 segundos, cubriendo el punto extra de actualización automática.

Listing 57: `dashboard.html`: refresco automático cada 2 s

```

function refrescar() {
    fetch("/panel").then(function(r){ return r.json(); })
        .then(pintar)
        .catch(function(){ document.getElementById("total").innerHTML =
            '<span class="mal">no se pudo contactar al nodo</span>'; })
        .finally(function(){
            document.getElementById("reloj").textContent = new
            ↪ Date().toLocaleTimeString("es-MX");
        });
}

refrescar();
setInterval(refrescar, 2000);

```

9 Despliegue en Google Cloud Platform

El cluster del mini banco se despliega en el proyecto `sd-final-eddndev` de Google Cloud Platform, sobre la zona `us-central1-c`. Son tres máquinas virtuales de la serie E2 (obligatoria para este proyecto): `nodo-1` actúa como líder en una `e2-standard-4` (4 vCPU), mientras que `nodo-2` y `nodo-3` son réplicas en `e2-standard-2` (2 vCPU). El rol de cada nodo no se fija en la configuración de la VM, sino que lo deduce el propio binario a partir de variables de entorno (ver `MainServer.java`), por lo que la imagen desplegada es idéntica en las tres máquinas.

9.1 Topología y firewall

El líder expone su API HTTP en una IP estática reservada (34.136.17.97), de modo que los clientes y el dashboard siempre encuentran la misma dirección aunque la VM se reinicie o se vuelva a crear. Las tres VMs escuchan el tráfico HTTP en el puerto 8080, valor por defecto del nodo según `MainServer.java`:

Listing 58: Puerto por defecto del nodo (`MainServer.java`).

```
int puerto = 8080;
if (args.length > 0) {
    try { puerto = Integer.parseInt(args[0]); }
    catch (NumberFormatException e) { System.err.println("Puerto invalido, usando
↪ 8080."); }
}
```

El firewall del proyecto se configura con dos reglas de intención distinta. El puerto `tcp:8080` se abre a Internet (0.0.0.0/0) para que la API y el panel sean accesibles desde fuera. La replicación entre nodos viaja por el puerto 9090 (valor por defecto de `BANCO_REPLICACION_PUERTO` en `MainServer.java`) y se restringe a la red interna: ese canal solo debe ser alcanzable por las propias VMs del cluster, no desde Internet.

Listing 59: Reglas de firewall: API pública, replicación interna.

```
gcloud compute firewall-rules create banco-http \
  --network=default --allow=tcp:8080 --source-ranges=0.0.0.0/0

gcloud compute firewall-rules create banco-replicacion \
  --network=default --allow=tcp:9090 --source-ranges=10.128.0.0/9
```

9.2 Despliegue sin SSH via Cloud Storage y systemd

El SSH directo a estas VMs falla por `StrictModes` de `sshd` sobre el directorio `home` persistido en el disco, así que el despliegue no depende de iniciar sesión en las máquinas. En su lugar, los artefactos (`banco.jar` y `alumnos.csv`) se publican en un bucket de Cloud Storage, y cada VM lleva un `startup-script` en su metadata que se ejecuta en cada arranque: instala el JRE, descarga los artefactos del bucket e instala y arranca un servicio de `systemd`.

Listing 60: Startup-script en metadata: JRE, artefactos y servicio.

```
#!/bin/bash
apt-get update -y && apt-get install -y default-jre
mkdir -p /opt/banco
gsutil cp gs://sd-final-eddndev-artefactos/banco.jar /opt/banco/banco.jar
gsutil cp gs://sd-final-eddndev-artefactos/alumnos.csv /opt/banco/alumnos.csv
cp /opt/banco/banco.env /etc/banco.env # variables de entorno por rol
systemctl daemon-reload
systemctl enable --now banco.service
```

El servicio `banco.service` corre el *fat jar* con `Restart=always`, de forma que el nodo se vuelve a levantar solo si el proceso muere. La configuración por rol se inyecta mediante un archivo `EnvironmentFile`, lo que evita codificar credenciales o topología dentro del jar.

Listing 61: Unidad `systemd` `banco.service` (`Restart=always`, puerto 8080).

```
[Unit]
Description=Mini Banco Distribuido
After=network-online.target

[Service]
EnvironmentFile=/etc/banco.env
ExecStart=/usr/bin/java -XX:+UseParallelGC -Xms8g -Xmx8g -XX:+AlwaysPreTouch -jar
    ↪ /opt/banco/banco.jar 8080
Restart=always
RestartSec=2

[Install]
WantedBy=multi-user.target
```

NOTA

Los *flags* de JVM del `ExecStart` suben el throughput del líder ~10 % bajo saturación (medido A/B con generador co-locado): en una ráfaga CPU-bound de 60s, `ParallelGC` evita el marcado concurrente de G1 que roba CPU, y `-Xms8g -Xmx8g -XX:+AlwaysPreTouch` fija el heap y toca las páginas al arrancar para no pagar `resize` ni fallos de página en caliente. El heap de 8 GB es **solo para el líder** (e2-standard-4, 16 GB de RAM); las réplicas (e2-standard-2, 8 GB) deben omitir `-Xmx8g` o usar un valor menor.

NOTA

Re-desplegar una versión nueva no requiere SSH: basta con volver a subir `banco.jar` al bucket y hacer un *reset* de la VM. El `startup-script` se ejecuta de nuevo en el arranque y trae el artefacto actualizado antes de levantar `banco.service`.

9.3 Configuración por variables de entorno

Las tres VMs comparten el mismo jar; lo que las diferencia es el conjunto de variables de entorno del `EnvironmentFile`. El secreto `BANCO_JWT_SECRET` (leído en `JWTUtil.java`) debe ser idéntico en los tres nodos para que un token emitido por cualquiera sea válido en los demás. La tabla resume las variables relevantes y donde las consume el código.

Variable	Funcion y origen
ALUMNOS_CSV	Ruta del dataset de cuentas; <code>MainServer.java</code> .
BANCO_JWT_SECRET	Secreto JWT compartido por los 3 nodos; <code>JWTUtil.java</code> .
BANCO_NODO_ID	Identificador del nodo (nodo-1/2/3); <code>MainServer.java</code> .
BANCO_PEERS	Peers del panel del lider (host:port); <code>MainServer.java</code> .
BANCO_LIDER_HOST	Host del lider que siguen las replicas; <code>MainServer.java</code> .
BANCO_GCS_BUCKET	Bucket del log durable, solo lider; <code>MainServer.java</code> .
BANCO_REACTORES	Numero de hilos de I/O del servidor; <code>BancoHttp.java</code> .
BANCO_CHECKPOINT	Ruta del punto de control local de la replica (def <code>checkpoint.bin</code> ; en prod <code>/opt/banco/checkpoint.bin</code>); <code>MainServer.java</code> .
BANCO_CHECKPOINT_SEG	Periodo en segundos del snapshot de la replica (def 3); <code>MainServer.java</code> .

El reparto del rol no se declara: lo decide la ausencia o presencia de `BANCO_LIDER_HOST`. El método `esLider()` de `MainServer.java` considera lider a todo nodo que no tenga esa variable, y `iniciarReplicacion()` ramifica en consecuencia.

Listing 62: El rol se infiere de `BANCO_LIDER_HOST` (`MainServer.java`).

```
private static boolean esLider() {
    String liderHost = System.getenv("BANCO_LIDER_HOST");
    return liderHost == null || liderHost.isBlank();
}
// ...
if (esLider()) {
    new ServidorReplicacion(CuentaRepository.get(), puertoRepl).iniciar();
} else {
    String liderHost = System.getenv("BANCO_LIDER_HOST");
    CuentaRepository repo = CuentaRepository.get();
    ClienteReplicacion cliente = new ClienteReplicacion(repo, liderHost, puertoRepl);
    // ...
    cliente.iniciar();
}
```

Por esa logica, el `EnvironmentFile` del lider (nodo-1) lleva `BANCO_PEERS` y `BANCO_GCS_BUCKET`, pero NO `BANCO_LIDER_HOST`; solo el lider activa el log durable en Cloud Storage (ver `iniciarAlmacenGcs()`). Las replicas (nodo-2 y nodo-3) llevan `BANCO_LIDER_HOST` apuntando al lider y carecen de bucket y peers.

Listing 63: `EnvironmentFile` del lider nodo-1 (`/etc/banco.env`).

```
ALUMNOS_CSV=/opt/banco/alumnos.csv
BANCO_JWT_SECRET=<secreto-compartido>
BANCO_NODO_ID=nodo-1
BANCO_PEERS=nodo-2:8080,nodo-3:8080
BANCO_GCS_BUCKET=sd-final-eddndev-log
BANCO_REACTORES=3
```

Listing 64: EnvironmentFile de una replica nodo-2 (/etc/banco.env).

```
ALUMNOS_CSV=/opt/banco/alumnos.csv
BANCO_JWT_SECRET=<secreto-compartido>
BANCO_NODO_ID=nodo-2
BANCO_LIDER_HOST=34.136.17.97
BANCO_CHECKPOINT=/opt/banco/checkpoint.bin
```

IMPORTANTE

BANCO_GCS_BUCKET se define solo en el lider. Aun asi, el codigo esta blindado: `iniciarAlmacenGcs()` se invoca unicamente cuando `esLider()` es verdadero, asi que una replica (que tiene `BANCO_LIDER_HOST`) ignora esa variable y nunca activa el log durable.

9.4 Operacion del punto de control en las replicas

El *startup-script* de `nodo-2` y `nodo-3` difiere del generico en dos detalles propios de la replica: define `BANCO_CHECKPOINT` (ruta absoluta y persistente del punto de control) en el `EnvironmentFile` y, antes de levantar el servicio, borra cualquier checkpoint heredado para garantizar un *bring-up* limpio. Sin ese borrado, una VM recreada sobre el mismo disco podria arrancar con un checkpoint viejo en lugar de partir del CSV.

Listing 65: Variante del startup-script de una replica (bring-up limpio).

```
rm -f /opt/banco/checkpoint.bin # estado fresco en cada arranque de la VM
echo "BANCO_CHECKPOINT=/opt/banco/checkpoint.bin" >> /etc/banco.env
systemctl enable --now banco.service
```

IMPORTANTE

Para bajar una replica en la demo usa `systemctl stop banco`: `systemd` envia `SIGTERM`, el *shutdown hook* escribe el checkpoint exacto y al hacer `systemctl start banco` la replica reanuda por secuencia. NO uses `kill`: por `Restart=always` el proceso revive de inmediato y la replica nunca permanece caída para demostrar la recuperacion.

10 Pruebas y calidad

La estrategia de pruebas del Mini Banco se apoya en un *harness* propio escrito en Java puro, sin JUnit ni ninguna otra biblioteca externa. Esta fue una decision deliberada: el proyecto no agrega dependencias de pruebas para no inflar el `pom.xml` ni el empaquetado, y porque las suites necesitan levantar componentes reales (servidor NIO, sockets TCP, repositorio concurrenente) que un framework no aporta. Todas las pruebas viven en el paquete `pruebas`, dentro de `src/test/java/pruebas/`, y comparten dos piezas de infraestructura: un corredor unico y un verificador minimo de aserciones.

10.1 El harness: TestRunner y MiniTest

El corredor `pruebas/TestRunner.java` invoca en secuencia el metodo estatico `run()` de cada suite, imprime un encabezado por suite y al final resume cuantas aserciones pasaron o fallaron. Si hubo al menos un fallo, termina con `System.exit(1)`, de modo que un script o un pipeline puede detectar la regresion por el codigo de salida.

Listing 66: TestRunner orquesta las suites y propaga el fallo via codigo de salida

```
System.out.println("= CuentaRepositoryTest =");
CuentaRepositoryTest.run();
// ... el resto de las suites ...
System.out.printf("%nResultado: %d pasadas, %d fallidas%n",
    MiniTest.pasadas(), MiniTest.fallidas());
if (MiniTest.fallidas() > 0) System.exit(1);
```

El verificador `pruebas/MiniTest.java` sustituye a las anotaciones y *assertions* de un framework con apenas tres metodos: `check()` para una condicion booleana y dos sobrecargas de `eq()` (una para `long` y otra para `Object`) que comparan un valor esperado contra el real. Lleva un par de contadores estaticos, `pasadas` y `fallidas`, e imprime el mensaje de la asercion que no se cumple en lugar de lanzar una excepcion, lo que permite que una suite reporte todas sus fallas en una sola corrida.

Listing 67: MiniTest: aserciones y conteo en Java puro

```
public static void check(boolean cond, String msg) {
    if (cond) {
        pasadas++;
    } else {
        fallidas++;
        System.out.println(" FALLO: " + msg);
    }
}

public static void eq(long esperado, long real, String msg) {
    check(esperado == real, msg + " (esperado " + esperado + ", real " + real + ")");
}
```


10.2 Como se ejecutan

Toda la batería se corre con un solo comando, `./ejecutar_tests.sh`, que primero compila con Maven y luego lanza el corredor con el classpath del jar mas las clases de prueba.

Listing 68: `ejecutar_tests.sh`: compila y corre el harness

```
mvn -q package
java -cp "target/banco.jar:target/test-classes" pruebas.TestRunner
```

NOTA

En total las once suites suman **110 aserciones** verdes. El conteo final lo imprime el propio `TestRunner` y, mientras todas pasen, el script termina con código de salida 0.

10.3 Que cubre cada suite

Las suites van de la lógica de dominio aislada hasta los componentes distribuidos ejercitados con red real. Cada una se cita por su archivo fuente.

- `pruebas/CuentaRepositoryTest.java` (15 aserciones): valida la transferencia básica en centavos, los casos borde (`MISMA_CUENTA`, `MONTO_INVALIDO`, `NO_ENCONTRADA`, `SALDO_INSUFICIENTE`) y, sobre todo, la invariante de conservación del dinero lanzando 8 hilos con 5000 operaciones cada uno (40 mil transferencias concurrentes) y comprobando que `saldoTotalCentavos()` no cambia. También comprueba que ese método *recalcula* la suma real recorriendo las cuentas: tras mover un saldo fuera de una transferencia que conserva, `saldoTotalCentavos()` detecta el descuadre, algo que un contador cacheado no podría reportar.
- `pruebas/AlumnosLoaderTest.java` (7 aserciones): carga un CSV temporal con `AlumnosLoader.cargar()` y verifica el número de filas, la numeración de cuentas base 1, el armado del propietario y la conversión exacta a centavos (por ejemplo 36180311 para 361803.11).
- `pruebas/ReplicacionTest.java` (11 aserciones): comprueba que el log de transacciones registra en orden y que una replica converge al líder reapiando `desde(0)`, incluido el escenario de recuperación incremental (catch-up) en que la replica se quedó en la secuencia 150 y pide solo de la 151 en adelante.
- `pruebas/ReplicacionTcpTest.java` (4 aserciones): la misma convergencia pero con sockets reales. Levanta `ServidorReplicacion` en un puerto efímero y un `ClienteReplicacion` que se conecta por TCP a 127.0.0.1; valida el catch-up inicial y que el *streaming* en vivo propague nuevas transferencias.
- `pruebas/ReplicacionMultiWriterTest.java` (4 aserciones): estresa la replicación con 8 hilos escribiendo miles de transferencias sobre pares disjuntos en un repo líder, mientras un lector drena el log con `registro().desde(estado)` y lo reapiando en una replica. Al final exige no solo la suma global conservada en ambos repos, sino la igualdad de saldo *por cuenta* líder == replica para todas las cuentas; sin la serialización del par `seq+log` bajo `seqLock` el lector se saltaría una secuencia aun no insertada y la replica divergiría.
- `pruebas/PuntoControlTest.java` (18 aserciones): valida el punto de control en disco de la replica con seis casos. En `roundTrip()` guarda con `guardarFinal` y comprueba que cargar res-

taura saldos, secuencia y `totalTransferencias` (igual a la secuencia). `sinArchivo()` confirma que cargar devuelve -1 y deja la secuencia en 0 (arranque desde el CSV) cuando no hay archivo, y `archivoCorrupto()` que devuelve -1 cuando los bytes son basura. `archivoTruncado()` corta un checkpoint valido por la mitad (cabecera buena, cuerpo a medias) y exige -1 con saldos y secuencia sin tocar: cargar parsea todo antes de mutar, de modo que un truncado nunca deja un estado mitad-CSV/mitad-checkpoint. `resumeE2E()` ejercita la reanudacion completa con sockets reales: la replica alcanza al lider, persiste su punto de corte, se reinicia desde su secuencia exacta y pide solo `x+1..` al lider, terminando con saldos iguales al lider y `totalTransferencias` igual a la secuencia final (sin doble aplicacion). `resetSiAdelantada()` simula una replica que viene de un checkpoint mas nuevo que el estado del lider rezagado: el lider ordena RESET, el callback `alResetear` recarga la base y la replica converge al lider en vez de quedar divergente.

- `pruebas/GeneradorCargaTest.java` (5 aserciones): corre el `GeneradorCarga` unos 2 segundos contra un nodo HTTP real en proceso (`BancoHttp.crear(0)`) y verifica que hubo lecturas y transferencias exitosas sobre la pila HTTP, que descubrio el numero de cuentas via `/stats` y que la invariante de suma se conserva.
- `pruebas/PanelTest.java` (8 aserciones): comprueba que `/panel` agrega el `/stats` local, el de un peer vivo y marca como caido un peer inalcanzable, ademas de confirmar que `dashboard.html` viaja en el jar y consume `/panel` de forma reactiva con `setInterval`.
- `pruebas/AlmacenGcsTest.java` (10 aserciones): prueba el log durable con un subidor falso (sin tocar GCS ni la red) que puede fallar las primeras N veces. Verifica persistencia en orden, reintentos tras fallos transitorios, el sembrado del contador con objetos existentes, el enganche al commit via observador y la recuperacion reaplicando el log.
- `pruebas/ParserHttpTest.java` (10 aserciones): ejercita el parser HTTP/1.1 del servidor NIO con sockets *crudos*, en los casos borde que un cliente de alto nivel no toca: dos requests por `keep-alive`, pipelining en un mismo segmento TCP, body partido en dos segmentos, respuesta 405 con `Content-Length: 0` que no rompe la conexion, query string ignorada, header `Authorization` sin distincion de mayusculas y ausencia de autorizacion (401).
- `pruebas/HandlersErroresTest.java` (18 aserciones): ejercita los manejadores *in-process* (construye una solicitud a mano, sin socket) para fijar los codigos de error. Cubre el cuerpo de `/transfer` (JSON malo, campos faltantes, monto no numerico, misma cuenta, monto cero o negativo y saldo insuficiente devuelven 400; cuenta inexistente, 404; la transferencia valida, 200); el header `Authorization` tolerante con el scheme `Bearer` sin distincion de mayusculas y con espacios extra (`bearer x`, `BEARER x`, `Bearer x` dan 200, y sin scheme o con token roto, 401); y el GET de un id numerico fuera del rango `int` o inexistente, que devuelve 404 (no 400).

Listing 69: `ParserHttpTest`: dos requests en una sola conexion keep-alive sobre socket crudo

```
try (Socket s = new Socket("127.0.0.1", puerto)) {
    OutputStream out = s.getOutputStream();
    enviar(out, "GET /stats HTTP/1.1\r\nHost: x\r\n\r\n");
    MiniTest.eq(200, leerResp(s.getInputStream())[0], "keep-alive: 1a request GET
    ↪ /stats -> 200");
    enviar(out, "GET /stats HTTP/1.1\r\nHost: x\r\n\r\n");
    MiniTest.eq(200, leerResp(s.getInputStream())[0], "keep-alive: 2a request en la
    ↪ MISMA conexion -> 200");
}
```

CONSEJO

La mezcla de pruebas de dominio puro (`CuentaRepositoryTest`) con pruebas de integración sobre red real (`ReplicacionTcpTest`, `GeneradorCargaTest`, `ParserHttpTest`) da confianza tanto en la lógica como en el transporte, sin pagar el costo de un framework externo.